

Let us start by discussing logic based AI techniques. In the early days of AI, binary logic was commonly used to develop applications. Prolog, one of the first logic programming languages, was used to develop binary logic based applications. Prolog's declarative style and logic programming paradigm made it ideal for implementing expert systems, which were some of the earliest applications of AI. We will cover more about Prolog in the next article in this series, where we will discuss non-Pythonic programming languages for AI. Therefore, our focus here is on fuzzy logic, which is a form of multi-valued logic. Unlike binary logic (Boolean logic), where the truth values of variables may only be the integer values 0 or 1, in fuzzy logic the truth values of variables may be any real number between 0 and 1. Python offers several libraries for implementing fuzzy logic, and one of the most popular and widely used one is scikit-fuzzy, which is built on top of NumPy and SciPy. It provides a range of functions for fuzzy inference, membership function generation, and rule optimisation. To install scikit-fuzzy in your system, you can execute the command `pip3 install scikit-fuzzy` in your terminal. The simple Python script shown below utilises the scikit-fuzzy library to implement fuzzy logic (line numbers are included for ease of explanation).

```
1. import numpy as np
2. import skfuzzy as fuzz
3. x = np.arange(0, 11, 1)
4. mflow = fuzz.trimf(x, [0, 0, 4])
5. mfmedium = fuzz.trimf(x, [2, 5, 8 ])
6. mfhigh = fuzz.trimf(x, [6, 10, 10])
7. val = fuzz.defuzz(x, np.fmax(mflow, np.fmax(mfmedium, mfhigh)), 'centroid')
8. print(val)
```

Now, let us go through the code line by line. Lines 1 and 2 import the libraries NumPy and scikit-fuzzy,

respectively. Line 3 creates a NumPy array `x` containing values from 0 to 10 with a step of 1. Lines 4 to 6 define three fuzzy membership functions, `mflow`, `mfmedium`, and `mfhigh`, using the function `trimf()` provided by the `fuzz` module of the `scikit-fuzzy` library. These functions represent triangular membership functions for low, medium, and high values of a variable. A triangular membership function is a fuzzy set membership function used to define the degree of membership of an element in a fuzzy set. It is defined by three parameters: the left, centre, and right boundaries. The triangular membership function takes on values between 0 and 1, with a maximum value of 1 at the centre and linearly decreasing values towards the left and right boundaries. This function is often used in fuzzy logic systems to represent concepts that have a natural range of values, such as temperature, speed, distance, etc. Line 7 calculates the defuzzified value. First, the maximum membership value of each membership function is calculated using the function `fmax()` of NumPy. Then, the overall maximum membership value of all the three membership functions is determined using the function `fmax()` again. Finally, the defuzzified value is calculated using the method `centroid()` and assigned to the variable `val`. Line 8 prints the calculated defuzzified value. On execution of this Python script, the value 5.0 will be printed on the terminal. Other Python libraries often used for implementing fuzzy logic include `fuzzylogic`, `pyfuzzylite`, etc.

There are numerous approaches to explore biologically inspired AI techniques. Among them, swarm intelligence and evolutionary computing stand out as key players in this field. Swarm intelligence is inspired by the collective behaviour of self-organised, decentralised systems observed in social insects and other animal societies. Its focus is on leveraging discovered solutions and refining them to enhance

performance. On the other hand, evolutionary computing encompasses a range of computational problem-solving techniques inspired by biological evolution, such as genetic algorithms, genetic programming, evolutionary strategies, and evolutionary programming. The wealth of options in biologically inspired computing can make it challenging to choose a specific approach. After careful consideration, I have decided that the focus of this discussion will be on genetic algorithms, an area that presented significant challenges for me during my postgraduate studies.

Genetic Algorithm (GA) is an optimisation technique that utilises the principles of natural selection and evolution to discover the optimal solution for a problem. GA operates on a population of individuals, each representing a candidate solution to the problem, which evolves through genetic operations like mutation, crossover, and selection. This iterative process of selection, replication, and mutation mirrors the mechanism of natural selection in biological evolution. Organisms with favourable traits are more likely to survive and pass on their genes, whereas those with unfavourable traits are less likely to do so. Similarly, genetic algorithms improve the population over generations, striving to find the optimal solution to the problem. GAs are highly efficient in exploring the search space of a problem and finding a good solution. They have proved useful in various domains, including scheduling, optimisation, and robotics.

Our familiar SciPy provides a wide range of optimisation algorithms, including those based on genetic algorithms. For example, the method `scipy.optimize.differential_evolution()` uses a type of genetic algorithm called differential evolution. However, the most popular Python framework for implementing genetic algorithms is DEAP (Distributed Evolutionary Algorithms in Python). To install